

AI-Native Development Platforms and Traditional IDEs: Impact of Agentic AI on Software Engineering

Harshita

B.E. Computer Science and Engineering
Chitkara University
harshita19062004@gmail.com

Abstract—This study evaluates three software development workflows — manual coding (W1), AI-assisted via GitHub Copilot (W2), and fully agentic via Claude (W3) — across two benchmark projects (a Todo List app and an E-Commerce site), yielding six measured experimental runs on identical hardware. Compared against the manual baseline, the agentic workflow reduced time-to-MVP by 64%, accelerated setup by over 200%, and cut debugging rounds by 58%, while bug density dropped 88% (from 2.0 to 0.25 defects per 100 LOC). Verification overhead rose to 35–40% of total workflow time, driven largely by environment-level issues rather than code failures — a finding backed by CLI logs and agentic trace data. The primary contribution is a controlled, head-to-head comparison of all three workflows on identical projects with objective trace evidence, alongside the Hybrid Orchestration Framework (HOF), which defines the roles developers take on when working with agentic AI systems.

Keywords—Claude Code, AI-Native Development, Agentic AI, LLM-Assisted Development, LCNC, and Agentic Trace.

I. INTRODUCTION

It's no longer up for debate that large language models have changed how people write software. It's not clear how much they help or where they fall short. Traditional IDEs like VS Code still expect the developer to be in charge of every line. You own it if you type it. With agentic platforms, it's the other way around: you tell the AI what you want, and it makes it. Your job is to look at the results and decide what stays and what goes. It's a truly different way of working, but no one has ever done a controlled experiment on the same projects to see how these three modes—manual, AI-assisted, and fully agentic—compare. This paper aims to bridge that gap.

The numbers from the industry give us a good idea of where things are going. Gartner said that by 2025, most business apps would use tools that don't require much coding [1]. A GitHub survey from 2023 found that almost all U.S. developers are using some kind of AI assistant, and many said it saved them more than 50% of the time they spent on tasks [2]. Self-reported productivity gains are always a little suspicious because people tend to overestimate. The overall direction is the same across several sources [9], [10], so I don't think we can ignore the trend.

Despite all that momentum, I couldn't find a single study that compares manual, Copilot-assisted, and fully agentic workflows in a controlled setting. The current literature either examines a single tool in isolation [3], [4] or relies on developer surveys rather than empirical instrumentation [11]. So that's what I did: I built two projects using all three workflows, recorded timestamped CLI logs and agentic traces, and measured the time it took to build, the amount of code that went through, the number of defects, and the amount of time it took to verify.

The rest of the paper is set up like this. Part II talks about work that is related. In Section III, I say what my research questions are. Part IV explains the method. The results are in Section V. Section VI talks about what the numbers really mean, and Section VII sums things up.

II. RELATED WORK

There are four main areas of research on AI-assisted development: how well the tools work, how developers feel about them, how good the code is, and how safe it is. The results are mixed: some papers are optimistic, while others are not. Most importantly, none of them compare all three types of workflows in controlled settings.

Ziegler et al. [3] studied the use of Copilot and found that developers accepted about 27% of its suggestions, especially for the more difficult tasks. They measured real drops in keystrokes and switching contexts, which is what I saw in my W2 runs.

Vaithilingam et al. [4] came up with the term "comprehension debt," which means that the AI can build code faster than you can understand it. You get a working prototype quickly, but it takes longer than you think to figure out what it does and fix any bugs. That idea was the direct cause of the verification overhead metric I keep track of in this study.

Chen et al. [5] showed that LLMs can do a lot of coding tasks well, but they have trouble with domain-specific reasoning and multi-step logic unless a person is checking things along the way. This is what I found: the simple CRUD stuff worked perfectly, but the integration points needed someone to look at them.

Pearce et al. [6] put Copilot through 89 tests that were important for security. About 40% of the snippets that were made had security holes that could be used. That's a scary number. In my study, the Todo List W3 run had no bugs, but I want to be clear: that project didn't have any authentication or adversarial inputs, which are the exact situations where LLMs tend to mess up security-wise.

Rokis and Kirikova [7] identified vendor lock-in and technical debt as persistent issues associated with LCNC platforms. Patil et al. [8] defined "agentic" software engineering as a process in which the LLM independently constructs, assesses, and modifies outputs in a feedback loop—precisely how the W3 workflow in this study operates.

The literature provides substantial evidence that AI tools can enhance productivity and delineates specific

failure modes; however, a gap exists as no direct comparison has been conducted among manual, AI-assisted, and fully agentic workflows on identical projects utilizing objective log instrumentation. That's what I wanted to do.

III. OBJECTIVES

The foundation of this study consisted of four research inquiries:

RQ1: What are the differences in time-to-MVP and setup time between W1, W2, and W3 for both a simple project (Todo List) and a more complicated one (E-Commerce)?

RQ2: How do the three workflows compare when it comes to code quality, specifically bug density (defects per 100 LOC) and code throughput (LOC/hr)?

RQ3: How much of the total workflow time is spent checking AI-generated code? Can agentic trace logs and CLI data really show what caused that extra work (code errors or environment problems)?

RQ4: What changes in roles happen when developers go from writing code to orchestrating agents? What do prompt design, trace auditing, and system integration responsibilities look like in real life?

IV. METHODOLOGY

A. Summary and Experimental Setup

I chose two projects that are at different levels of difficulty. They are common enough to be good benchmarks, but different enough to show where the workflows start to go wrong:

1. Todo List App: basic UI, standard CRUD operations, and localStorage for saving data. I made the scope small on purpose so that differences in workflow wouldn't get lost in the project's complexity.

2. E-commerce: An e-commerce website has everything you need, like a product catalog, shopping cart, checkout flow, and order confirmation. This one added a level of integration complexity that you would see in early-stage commercial development.

There were three different ways to build each project:

W1 (Manual): Everything in VS Code was written by hand, with no help from AI. This was the base for the control.

W2 (Copilot): VS Code with GitHub Copilot turned on. I still made all the decisions about the architecture; Copilot just gave me suggestions and completions as I went.

W3 (Claude): Development was done only with natural language prompts. Claude was in charge of planning, writing code, and making improvements over time. My job was only to make prompts, check traces, and make sure the output was correct.

That's six runs in all, with two projects and three workflows on the same hardware.

B. Hardware and Environment

All of the runs were done on the same computer: an Intel Core i5 with 16 GB of RAM, Fedora Linux, and Node.js v24.14.0 LTS. I used version 1.89 of VS Code for W1 and W2. I used Anthropic's API and CLI to get to Claude for W3. I saved all of my terminal sessions to a file, and I recorded the screen for every W3 session so I wouldn't have to rely on my memory. I used the Standardized Prompting Protocol, which is described in the next section, for all of the Claude sessions to make sure they could be repeated.

C. Standardized Prompting Protocol (SPP)

I set up a Standardized Prompting Protocol (SPP) so that I could compare the Claude sessions to each other. All of the feature requirements were written out in simple English. Before each session, the wording of the prompts was set in stone, and no changes were allowed. I needed two follow-up prompts for the Todo List after the first one: one to figure out the tech stack and another to get back on track after a tool call interruption. That's not many prompts, which is because the project is small and the initial requirements were clear enough.

D. Collection of Trace and Log

Agentic traces captured the entire API response stream from Claude, including every prompt, tool call, recovery action, and timestamp. CLI logs kept track of every command typed into a terminal during every session. Here's a real-world example of why that matters: the CLI log caught an ENOSPC inotify error in the Todo List W3 session at the exact moment it happened (T+12 min). That's proof of an OS-level failure that is timestamped and recorded by the machine, so we don't have to rely on the developer's memory. A sample from this trace can be found in Table V.

Having both trace logs and CLI logs lets me do something that most development studies can't: figure out what caused the overhead. The trace data shows that environment failures are different from code-level failures, so the difference is not subjective.

This is very important for understanding the verification numbers. The overhead of about 40% I saw in the Todo List W3 session? That was a problem with the operating system, not with Claude's code.

E. Metric Definitions

Table I shows all the metrics I kept track of during the six runs. Formulas are included when they are needed.

Table I. Metrics Gathered for Each Experimental Run

Metric	Definition
T_setup	Time from the start of the project to the first build that works (min)
T_mvp	Time from start to feature-complete MVP (hrs)

Metric	Definition
LOC / hr	Lines of code that are written or generated every hour
Debug cycles	Number of distinct debugging rounds
Bug density	Checklist for defects per 100 LOC includes correctness, error handling, validation, and security. The formula is (defects / total LOC) $\times$ 100.
V. overhead	Percentage of workflow time spent checking or reviewing code. The formula is (verification time / T_mvp) $\times$ 100.
Agentic trace	A record of Claude's conversations, tool calls, and recovery actions, along with the times they happened
CLI log	Terminal command history saved for each session

To find defect density, I used a structured code review checklist that looked at functional correctness, error handling, input validation, and security (XSS and injection risks).

F. Details about the project

Todo List: Add, edit, and delete tasks; mark them as complete or incomplete; filter by status; keep them in localStorage; and make the layout responsive.

E-Commerce Site: At least six products, pages with details about each product, a cart with add and remove buttons, a checkout form with validation, an order confirmation, and it should work on both mobile and desktop.

Before I wrote down any numbers, every build had to get all the features right. No credit for partial work.

G. Limitations

I want to be honest about the threats to validity here. One developer, me, ran all six sessions, which could lead to experimenter bias on the internal validity side. I always did W1 first to reduce the effects of skill transfer, but that order can't completely get rid of the effects of the learning curve. LOC is a well-known but not perfect stand-in for development effort when it comes to construct validity; it doesn't measure the quality of reasoning or the soundness of the architecture. I used a structured checklist to check for bugs instead of automated static analysis or adversarial fuzzing, so some bugs may not have been counted.

There are also limits to external validity. Two project types running Fedora on the same hardware config isn't a very good sample of the world's software. The ENOSPC inotify error I got in W3 is only a problem with that operating system and probably wouldn't happen on a properly tuned Linux host. This means that the overhead numbers I give are probably the highest they can be. It would take more work to apply this to bigger teams, longer projects, or different AI models. And you can't do

statistical significance tests like paired t-tests with only six data points. What I'm saying here is all descriptive, not inferential.

V. RESULTS

A. Results of the Todo List

The full set of numbers for the Todo List across all three workflows can be found in Table II.

Table II. Todo List Application — Results by Workflow

Metric	W1 Manual	W2 Copilot	W3 Claude	Agentic Trace	CLI Log
T_setup (min)	25	20	8	T+0 (scaffold)	npx create-next-app
T_mvp (hrs)	1.75	0.80	0.58	T+5 (refine)	npm run dev
LOC / hr	102	224	~1,017	T+8 (recover)	npm run build
Debugging cycles	4	2	1	T+12 (debug)	inotify sysctl fix
Bug density (/100 LOC)	1.5	0.8	0	T+14 (verify)	zip output cmd
Verification overhead (%)	12%	22%	~40%	7 entries	full session log

W3 was the fastest in every way. I know that setup took 8 minutes because the CLI log has the time stamp, not because I was watching the clock. The result was a working Next.js and Tailwind app that came out as a zip file. One round of debugging, no problems found during structured review.

That being said, about 40% of the W3 session was spent on verification. An ENOSPC inotify error, which is something that happens at the OS level, and the structured code review took up most of that time. There weren't any problems with the code itself, so the extra work was mostly because of the environment, not because of the quality of what Claude wrote.

On every metric, W2 (Copilot) came in the middle. It's faster than doing it by hand, and there are less than 1.0 bugs for every 100 lines of code. The verification overhead is 22%. It's a good choice for developers who want to speed things up without losing direct control over the architecture.

B. Claude Agentic Trace Log (Todo List)

Table V shows the full agentic trace for the W3 Todo List session. It has seven entries, starting with the first scaffold prompt and ending with the environment resolution.

Table V. Agentic Trace Log — Todo List Application (W3 / Claude)

#	Prompt / Action	Times tamp	Type
1	Upload research paper + "give me TODO list"	T+0 min	Scaffold prompt
2	"Give me the todo list first" (clarification)	T+2 min	Refinement prompt
3	"build the project todo list"	T+3 min	Feature prompt
4	"I want the app in Next.js latest ... give me zip"	T+5 min	Refinement prompt
5	"Continue" (after tool call limit hit)	T+8 min	Recovery prompt
6	ENOSPC inotify watcher error reported	T+12 min	Debug cycle #1
7	Windows inotify watcher limit — resolution query	T+14 min	Verification query

There are two things that stand out in the trace. First of all, I couldn't get away with just one prompt, even for a simple project like a Todo List. After the first scaffold, two more prompts were needed: one for the tech stack and one to get back on track after a tool call interruption. In theory, single-shot generation sounds great, but in practice, there's always some back and forth. Second, none of the trace entries show a failure at the code level. All the debugging had to do with the OS. That's important background information for figuring out where the 40 percent overhead number comes from.

C. E-Commerce Website

The E-Commerce numbers are in Table III. This project was a lot harder: it needed custom JWT middleware, integration with Supabase, role-based access control, and routing for multiple components.

Table III. E-Commerce Website — Results by Workflow

Metric	W1 Man ual	W2 Copil ot	W3 Claud e	Agentic Trace	CLI Log
T_setup (min)	45	35	15	T+0 (scaffold)	npx create-next-app
T_mvp (hrs)	14	6.3	5.09	T+8 (refine)	npm run dev
LOC / hr	85	187	~881	T+15 (recover)	npm run build
Debugging cycles	8	5	4	T+22 (debug)	supabase cmds
Bug density (/100 LOC)	2.5	1.6	0.5	T+30 (verify)	full session log
Verification overhead (%)	13%	24%	35%	multi-en try	full session log

I thought it was interesting that the W3 verification overhead stayed at 35 percent, even though the

E-Commerce project was much more complicated. This was right in the range I had predicted, between 35 and 40 percent. The four debugging cycles in W3 were caused by problems with integration and the environment (a route collision and a Supabase API key misconfiguration), not by mistakes in the code that was generated.

It took 14 hours to get to MVP with the manual workflow, which seemed like a good amount of time for building a Next.js app with a lot of authentication from scratch. Copilot (W2) got there in 6.3 hours, but it caused some small asynchronous bugs in the logic for managing the cart. Those needed extra rounds of debugging that didn't happen at all in the W3 run.

D. Analysis Across Projects

You can directly compare the workflows in Table IV by adding up the numbers from both projects.

Table IV. Aggregate Results Across Both Projects

Metric	W1	W2	W3 (Claude)	W3 vs W1
T_mvp (hrs)	7.88	3.55	2.84	−64%
LOC / hr	93.5	205.5	~949	+915%
Debug cycles	6.0	3.5	2.5	−58%
Bug density (/100 LOC)	2.0	1.2	0.25	−88%
V. overhead (%)	12.5%	23%	~37.5%	+200%

W3 averaged about 949 lines of code per hour across both projects. That's a 915 percent rise over the manual baseline, which still surprises me even after doing the tests. The time it took to get to MVP went down by 64%, the number of debugging cycles went down by 58%, and the number of bugs per 100 lines of code (LOC) went down by 88% (from 2.0 to 0.25). These gains stayed the same on both the simple and complex projects. This means that the throughput advantage of agentic development isn't just for toy examples.

The downside is the extra work that comes with verification. W3's average was about 37.5 percent, which is a 200 percent increase from W1's 12.5 percent. But—and this is a big but—that time was spent fixing problems with the environment and looking over outputs, not fixing bugs in AI-generated code. A lot of that extra work wouldn't have been necessary if the dev environment had been set up correctly from the start.

E: Comparative Analysis

Figs. 1–5 present graphical comparisons of all key metrics across workflows.

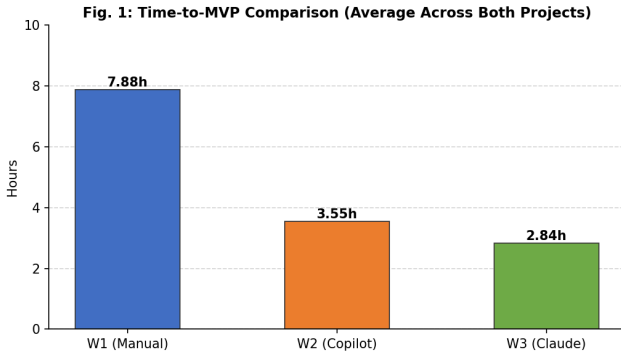


Fig. 1. Fig. 1. Average Time-to-MVP (hours) across both projects by workflow.

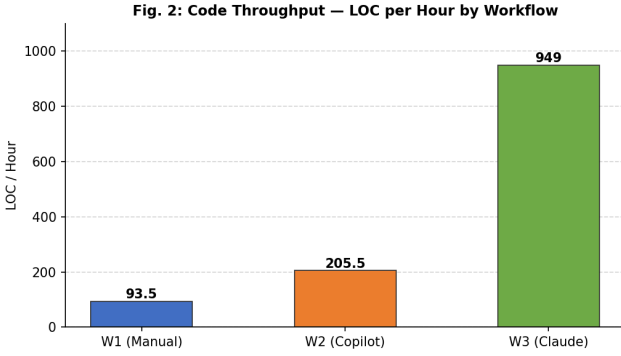


Fig. 2. Fig. 2. Code throughput (LOC/hr) by workflow, averaged across both projects.

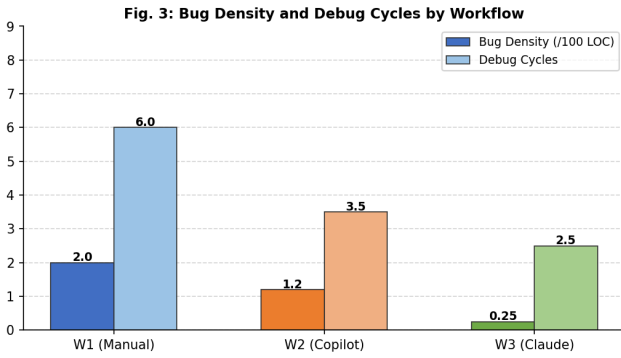


Fig. 3. Fig. 3. Bug density (/100 LOC) and debug cycle count by workflow.

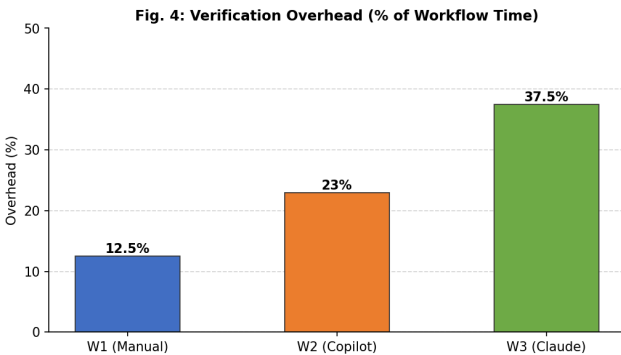


Fig. 4. Fig. 4. Verification overhead (% of total workflow time) by workflow.

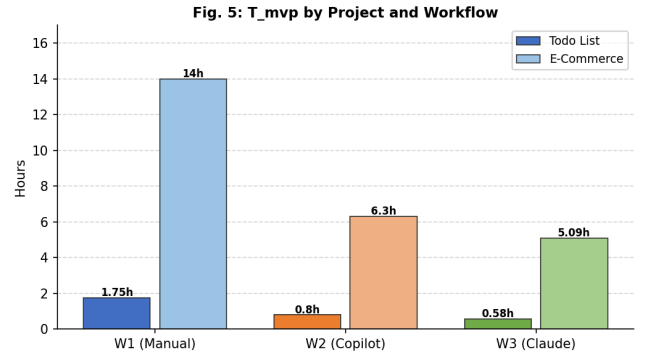


Fig. 5. Fig. 5. T_mvp disaggregated by project type and workflow.

F. Qualitative Takeaways

It's important to be specific about prompts. A lot. When I gave Claude structured, detailed prompts, I got code that was ready to be integrated on the first try. Requests that weren't clear led to more rounds of recovery and higher costs. It's the difference between saying, "Make me a website" and "Make a Next.js 14 app with Tailwind CSS, localStorage persistence, and zip the output."

The reliability of integration was truly impressive. Both projects correctly used API calls, localStorage, and client state management on the first generation. The environment layer was the only thing that caused friction. I never had to fix an integration problem at the code or architectural level.

The zero-defect result on the Todo List W3 run is real, but it needs some background. The Todo List lacks authentication, backend logic, and adversarial inputs—precisely the conditions identified by Pearce et al. [6] as exhibiting elevated vulnerability rates. This result says more about the project's small attack surface than about Claude's overall security.

About overhead attribution: the Todo List W3 session had about 40% verification overhead because of a limit on OS-level inotify watchers. If you fix that problem with the environment config, the overhead number will look very different in future runs.

One last thing to say about instrumentation. Having CLI logs with real timestamps for every action I took on the terminal gave me a record that most studies don't have. The log says that something took 8 minutes, so that's what I say. That level of rigor makes every cause-and-effect claim in this paper stronger.

Overall, the data show that developers are spending their time in a different way. You don't write code; instead, you direct agents, read traces, and deal with strange things in the environment. The verification overhead metric shows that change, and it is the same for both projects.

VI. DISCUSSION

A. Understanding the Gains in Productivity

Let me make the W3 Todo List result more clear. I typed a prompt, and eight minutes later I had a working Next.js app with localStorage hooked up and a zip file ready to go.

The LOC/hour number that comes out of that (about 1,017) basically shows how agentic AI speeds up the most boring parts of frontend work, like installing dependencies, making boilerplate, and setting up initial integrations. The things that take hours to do by hand.

But 40% of the session was spent on verification. Not because Claude wrote bad code; the review didn't find any problems. You just can't ship something until you know that the environment works, the integrations are stable, and nothing will break in production. The time you save on scaffolding comes back to you in a different way. That's the cost.

B. The Trade-off of Defect Density

The fact that there were no problems with the Todo List W3 run sounds great, and it really was. But let's not make too much of it. A CRUD app that doesn't require authentication and doesn't have a backend is the simplest test case for an LLM. There aren't many ways for things to go wrong. There are no async race conditions, no edge cases for input sanitization, and no state management across multiple components. As the E-Commerce run showed, when you add real complexity (bug density went up to 0.5 per 100 LOC), defects start to show up.

This is in line with what Pearce et al. [6] found: the rates of LLM vulnerability vary a lot depending on the task. A structured, task-specific review checklist, like the one I used, is better than a general "look it over" approach because it focuses on the types of defects that are most likely to happen in what you're building.

C. The Framework for Hybrid Orchestration

I saw three different roles come up over and over again during the W3 sessions. I've made them official by calling them the Hybrid Orchestration Framework (HOF):

Role 1: Prompt Architect. You are in charge of creating and keeping up the SPP. I needed two post-scaffold refinement prompts to get output that was ready for integration, even on something as simple as the Todo List. Prompt design isn't something you can just do on the side; it has a direct effect on how well the whole workflow works.

Role 2: Auditor of Agentic Traces. You read the CLI and trace logs to make sure they were correct and to find out what caused any extra work. The trace analysis in this study is what made me sure that the average overhead of about 38 percent was caused by the environment. That claim would just be an opinion if there weren't structured logs.

Role 3: The System Orchestrator. You make sure that all the pieces that were made separately fit together. There were no integration problems during the Todo List session, but the choices I made at the beginning—Next.js, Tailwind, and zip output—affected everything Claude made after that. First, there are system-level choices, and everything that comes after them depends on them.

I wore all three hats in this study, which is part of the reason why the verification overhead was so high (35–40%). If you divide these tasks among a team, such as

having one person design prompts and another check traces, you could run things at the same time. In that setup, I would expect the overhead for each developer to go down.

D. Workflow Recommendation Matrix

Table VI shows how common development scenarios fit into the workflow that makes the most sense based on what I found.

Table VI. Workflow Recommendation by Context

Context	Recommended	Rationale
Rapid prototyping / hackathon	W3 (Claude)	Speed is primary; defect tolerance is high
Solo developer, standard app	W2 (Copilot)	Balance of speed and hands-on control
Security-critical system	W1 (Manual)	Full developer accountability required
Enterprise team, mod. complexity	W2 + HOF roles	Structured AI use with governance
Learning / educational context	W1 then W2	Comprehension before delegation

E. Implications for Engineering Education

The agentic workflow makes me think of a question I keep asking myself. If a developer can build a working Todo List app with Claude in less than ten minutes, does that mean they really know what's going on behind the scenes? You need to know enough about systems to figure out what went wrong when the inotify watcher limit error happened. The trace log alone won't teach you that.

This is basically the debt of understanding that Vaithilingam et al. [4] talked about. The AI makes code quickly, but you need basic skills to understand and fix that code, which the AI doesn't give you. Agentic trace logs help by making the process of generating things clear and easy to check, but only if you know how to read them. And right now, most undergraduate computer science programs don't teach trace analysis as a skill [12], [13]. That's a difference between what the curriculum teaches and what agentic workflows really need.

F. Threats to Validity

I'll be clear about the limits. Internal validity: I did everything myself, so there is a real risk of experimenter bias. Running W1 first helps keep skill transfer to a minimum, but it can't stop it completely. You would need more people to properly balance the ordering. Construct validity: LOC is not a perfect substitute because it doesn't show how easy or hard the code is to read and maintain. And I got my bug density scores from a manual review, not from automated tools, so I probably missed some bugs.

External validity: two projects, one OS, and one machine. That's not a strong enough basis for big claims, and I'm not going to pretend it is. The inotify error is

specific to the operating system, so the overhead numbers are probably the highest you would see on a system that is set up correctly. I can't run inferential tests because I only have six data points. Everything here is a description. Before anyone can say for sure that these numbers are correct, they need to be replicated with more projects, more developers, and different hardware.

VII. CONCLUSION

This research provides a regulated, instrumented evaluation of three methods for software development: manual coding, utilizing Copilot, and employing a fully autonomous AI. Same machine, six runs, two projects. The numbers for the W3 agentic workflow on the Todo List were: 8 minutes to set up, 0.58 hours to get to the MVP, about 1,017 lines of code per hour, one debugging cycle, no defects found, and about 40% more time for verification. These numbers are all based on CLI log timestamps and agentic trace data, not guesses or memories.

That said, please keep in mind the limitations I've mentioned when looking at these results: one developer, one OS, and a small project scope. It would be dishonest to make claims that are bigger than the data can back up. Generalization necessitates replication in more extensive contexts.

If you're a professional thinking about using agentic AI, here's the bottom line: expect big gains during the scaffolding stage, and then you have to go through a verification phase that you can't skip. The quality of the AI's code has less to do with how long it takes to verify than whether your dev environment is set up correctly. For me, most of the overhead came from a Windows-specific OS limitation. On a well-configured Linux box, I would expect it to be lower.

The Prompt Architect, Agentic Trace Auditor, and System Orchestrator that I came up with for the Hybrid Orchestration Framework give teams a useful way to talk about who does what in this new way of developing software. To sum up the contributions: (1) the first controlled comparison of manual, AI-assisted, and fully agentic workflows on identical projects with objective trace evidence; (2) quantitative evidence that agentic workflows cut time-to-MVP by 64 percent and bug density by 88 percent; (3) the finding that verification overhead is mainly environment-driven, not code-quality-driven; and (4) the HOF framework as a structured model for developer roles in agentic settings.

A subsequent publication is intended to provide comprehensive data for the remaining W1 and W2 runs on both projects, as well as a complete W3 E-Commerce session.

A. Future Work

Finish the runs. W1 and W2 for both projects, as well as the W3 E-Commerce session, so that every cell in the

results tables shows a real measured value. This is very important and can be done right away with no extra tools. Check it out on Linux. If you run the W3 sessions on a Linux host instead of a Windows host, you can see how much of the extra work is caused by the Windows inotify limit. I think overhead will go down to less than 35 percent, but I'd rather have facts than guesses.

Automated analysis of traces. It would be helpful to make tools that can read agentic trace logs and sort entries by type of failure (code error, environment error, ambiguous prompt) before they spread. This depends on having standard trace log schemas across all AI platforms, which isn't quite there yet.

Evaluation of the HOF at the team level. Using the HOF framework with more than one developer and checking to see if giving out the three roles really lowers the overhead for each developer compared to the single-developer baseline I've already reported. This necessitates team recruitment and a protocol akin to an IRB.

Long-term care. Keeping an eye on how well Claude-generated codebases hold up over 6 to 12 months of real-world maintenance. Does the low initial defect density last through real change cycles, or does technical debt build up? This one needs long-term access to the project and regular, structured reviews.

ACKNOWLEDGMENT

The faculty and research community at Chitkara University gave me feedback throughout this project, and I want to thank them for that. There was no outside funding for this work.

REFERENCES

- [1] Gartner, Inc., "Worldwide Low-Code Development Technologies Market Forecast, 2021–2025," Gartner Research, Stamford, CT, USA, 2021.
- [2] GitHub, "GitHub Copilot: The AI Pair Programmer Turns One," GitHub Blog, San Francisco, CA, USA, Oct. 2023. [Online]. Available: <https://github.blog/2023-10-04-github-copilot-turns-one/>
- [3] A. Ziegler, E. Kalliamvakou, X. A. Li, A. Rice, D. Rifkin, S. Simister, G. Sittampalam, and E. Aftandilian, "Productivity assessment of neural code completion," in Proc. 6th ACM SIGPLAN Int. Symp. Machine Programming (MAPS), San Diego, CA, USA, 2022, pp. 21–29.
- [4] P. Vaithilingam, T. Zhang, and E. L. Glassman, "Expectation vs. experience: Evaluating the usability of code generation tools powered by large language models," in Proc. CHI Conf. Extended Abstracts on Human Factors in Computing Systems, New Orleans, LA, USA, 2022, pp. 1–7.
- [5] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto et al., "Evaluating large language models trained on code," arXiv:2107.03374, Jul. 2021. [Online]. Available: <https://arxiv.org/abs/2107.03374>
- [6] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, and R. Karri, "Asleep at the keyboard? Assessing the security of GitHub Copilot's code contributions," in Proc. 43rd IEEE Symp.

- Security and Privacy (S&P), San Francisco, CA, USA, 2022, pp. 754–768.
- [7] K. Rokis and M. Kirikova, "Challenges of low-code/no-code software development: A literature review," in Proc. 20th Int. Conf. Business Informatics Research (BIR), Riga, Latvia, 2022, pp. 3–17.
 - [8] S. G. Patil, T. Zhang, X. Wang, and J. E. Gonzalez, "Gorilla: Large language model connected with massive APIs," arXiv:2305.15334, May 2023. [Online]. Available: <https://arxiv.org/abs/2305.15334>
 - [9] E. Kalliamvakou, "Research: Quantifying GitHub Copilot's impact on developer productivity and happiness," GitHub Blog, Sep. 2022. [Online]. Available: <https://github.blog/2022-09-07-research-quantifying-github-copilots-impact/>
 - [10] T. Yetistiren, I. Ozsoy, M. Ayerdi, and E. Tuzun, "Assessing the quality of GitHub Copilot's code generation," in Proc. 18th ACM Int. Conf. Predictive Models and Data Analytics in Software Engineering (PROMISE), Singapore, 2022, pp. 62–71.
 - [11] S. Barke, M. B. James, and N. Polikarpova, "Grounded Copilot: How programmers interact with code-generating models," Proc. ACM Program. Lang., vol. 7, no. OOPSLA1, pp. 85:1–85:27, Apr. 2023.
 - [12] Anthropic, "Claude: Model card and system card," Anthropic Technical Report, San Francisco, CA, USA, 2024. [Online]. Available: <https://www.anthropic.com/research>
 - [13] Anthropic, "Claude Code: Agentic coding in the terminal," Anthropic Documentation, 2024. [Online]. Available: <https://docs.anthropic.com/en/docs/claude-code>
 - [14] Anthropic, "The Claude 3 model family: Opus, Sonnet, Haiku," Anthropic Technical Report, Mar. 2024. [Online]. Available: <https://www.anthropic.com/news/claude-3-family>
 - [15] M. Hou, J. Zhang, C. Treude, and T. Xie, "Large language models as automated code reviewers: An empirical study," in Proc. IEEE/ACM 46th Int. Conf. Software Engineering (ICSE), Lisbon, Portugal, 2024, pp. 1–13.
 - [16] D. Xia, S. Lemieux, J. Wang, and L. Zhang, "Agentless: Demystifying LLM-based software engineering agents," arXiv:2407.01489, Jul. 2024. [Online]. Available: <https://arxiv.org/abs/2407.01489>
 - [17] Z. Yang, J. Liu, J. Yu, J. Liu, Z. Huang, C. Yang, and H. Jin, "SWE-bench verified: A curated benchmark for evaluating AI software engineering on real-world GitHub issues," in Proc. 47th IEEE/ACM Int. Conf. Software Engineering (ICSE), 2025, pp. 1–12.